REPORTE TAREA 1
ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Bárbara González Chiguay

10 de abril de 2026

01:39

1. Entorno experimental

Para la realización de los experimentos y el análisis de desempeño de los algoritmos, se utilizó el entorno detallado a continuación:

- **Hardware:** Procesador AMD Ryzen 5 4500U (2375 MHz, 6 núcleos físicos y lógicos) con 8,00 GB de memoria RAM física instalada.
- **Software:** Sistema operativo Microsoft Windows 11 Home Single Language. Las implementaciones en C++ fueron compiladas con g++ (MinGW-W64) versión 12,2,0.
- **Análisis de Datos:** Se utilizó el lenguaje Python versión 3,9,7 (distribución Anaconda) junto con las librerías `pandas` y `matplotlib` para el procesamiento de métricas y generación de gráficos.

1.1. Datasets y Metodología

Los datos utilizados para las pruebas corresponden a los casos definidos en el enunciado de la tarea:

- **Ordenamiento:** Arreglos de números enteros generados aleatoriamente con tamaños $N \in \{10^1, 10^3, 10^5, 10^7\}$.
- **Matrices:** Matrices cuadradas de dimensiones $N \times N$, con $N \in \{2^4, 2^6, 2^8, 2^{10}\}$.

Las mediciones brutas se almacenan en la carpeta `data/measurements/`. Cada archivo de resultados sigue una estructura de tres columnas: la primera representa la dimensión del problema (N), la segunda el tiempo de ejecución (segundos) y la tercera la memoria RAM ocupada (KB).

2. Resultados y Análisis

2.1. Análisis de Algoritmos de Ordenamiento

En el siguiente gráfico se observa que para $N = 10^7$, QuickSort se comporta como un algoritmo cuadrático $O(n^2)$. Esto confirma que la elección del pivote llevaron al algoritmo a su peor caso teórico, lo cual es impactante, ya que, hasta $N = 10^3$, QuickSort es incluso más rápido que MergeSort.

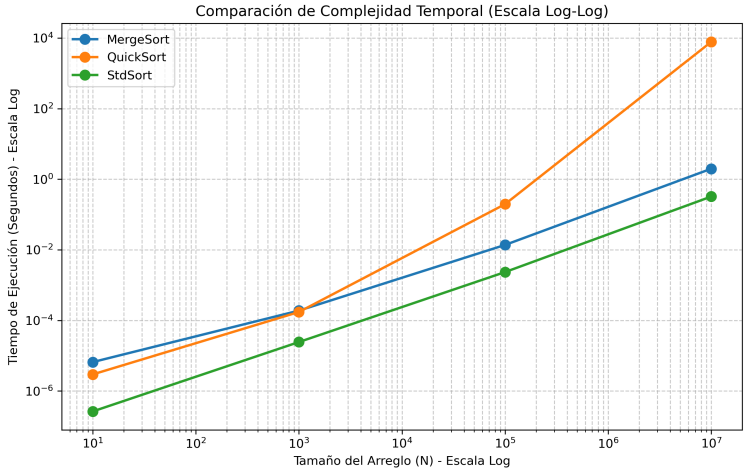

Figura 1: Comparación de complejidad temporal: Tiempo vs Tamaño

Por otro lado, tenemos el gráfico que compara el tamaño del arreglo con la memoria utilizada por cada algoritmo. Podemos notar que QuickSort es un algoritmo que trabaja como in-place, ya que, su consumo de memoria crece muy poco. Mientras que stdsort, siendo el más veloz, resulta ser el más costoso en términos de memoria RAM

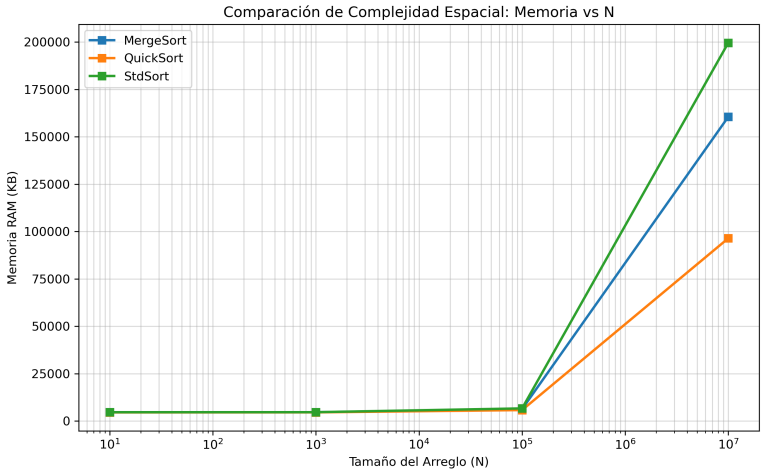

Figura 2: Comparación de complejidad espacial: Memoria vs Tamaño

2.2. Análisis de Multiplicación de Matrices

A continuación se presenta el gráfico de tiempo vs tamaño de los algoritmos de multiplicaion de matrices. El análisis muestra que el método Naive presenta un mejor desempeño temporal que Strassen para todas las dimensiones utilizadas en el experimento. Esto se atribuye al elevado de la recursión y la gestión de memoria en Strassen, sugiriendo que el beneficio de su menor complejidad asintótica ($O(n^{2,81})$) solo es apreciable en dimensiones significativamente mayores a las evaluadas.

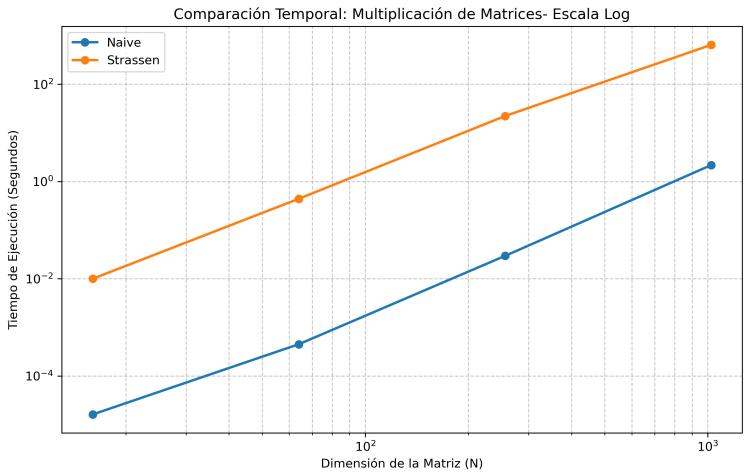

Figura 3: Comparación temporal: multiplicación matrices.

Por último, se presenta el gráfico que contiene el análisis de consumo de memoria de cada algoritmo de multiplicación de matrices. En términos de complejidad espacial, el experimento revela que el algoritmo de Strassen tiene un gasto de memoria muy superior al método Naive, alcanzando un pico de aproximadamente $55,000$ KB para matrices de 1024×1024. Este comportamiento es consistente con la naturaleza recursiva del algoritmo, representando un compromiso desfavorable del consumo real de recursos del sistema."

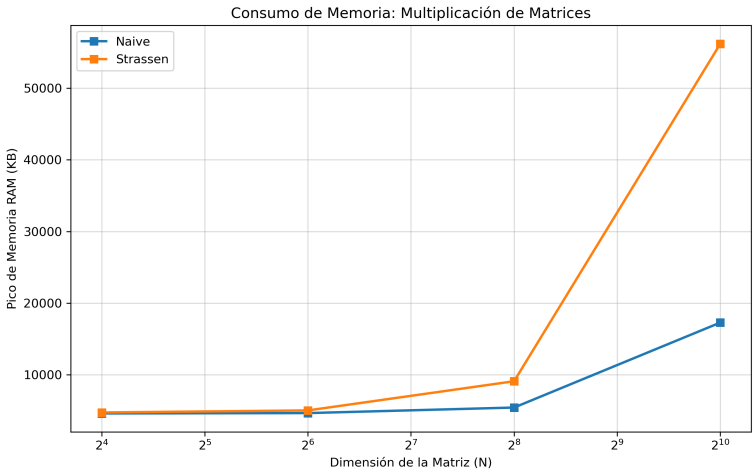

Figura 4: Consumo de memoria: Multiplicación de matrices

3. Conclusiones

El análisis experimental permitió validar la relación entre la teoría asintótica y el desempeño real del hardware. En el ámbito de ordenamiento, se observó que QuickSort es altamente sensible a la distribución de los datos, degradándose a una complejidad cuadrática para $N = 10^7$, mientras que MergeSort y std::sort mantuvieron la estabilidad esperada de la clase $O(n \log n)$. Respecto a la multiplicación de matrices, el método Naive resultó ser más eficiente que Strassen tanto en tiempo como en memoria para dimensiones de hasta 1024×1024. Esto demuestra que el elevado overhead recursivo y la gestión de memoria dinámica de Strassen superan su beneficio teórico en el rango evaluado, confirmando la importancia de considerar las constantes multiplicativas y el uso de recursos físicos al seleccionar un algoritmo en la práctica de la ingeniería.